

CSCE 633 MACHINE LEARNING · MODULE 04 · LECTURE NOTES

Regularisation and Model Selection

Spending capacity carefully, and choosing without cheating.

MODULE OUTCOMES

- Explain L2 and L1 regularisation and what each produces.
- Explain why L1 gives sparsity, geometrically.
- Implement cross-validation correctly, including preprocessing.
- Explain the three-way split and why the test set is spent once.
- Compare hyperparameter search strategies.

1 Penalising complexity

$$\text{Ridge: } \|Xw - y\|^2 + \lambda \|w\|^2$$

$$\text{Lasso: } \|Xw - y\|^2 + \lambda \|w\|_1$$

λ is the capacity dial of Module 03 §3, made explicit. $\lambda \rightarrow 0$ **recovers ordinary least squares**; $\lambda \rightarrow \infty$ **drives every coefficient to zero**. It is chosen by cross-validation (§2) and **never by looking at the test set**. **Ridge has a closed form** — $(X^T X + \lambda I)^{-1} X^T y$, and the added λI incidentally fixes the conditioning problem of Module 02 §1 — **while lasso does not, because the L1 norm is not differentiable at zero**. That non-differentiability is not an inconvenience to be worked around; it is precisely the mechanism that produces zeros.

WHY L1 GIVES SPARSITY, GEOMETRICALLY

Read the penalty as a constraint: minimise the loss subject to the coefficient vector lying within a region whose size is set by λ . The solution is where the expanding loss contours first touch that region.

The L2 constraint region is a sphere — a smooth surface with no distinguished points. The contact point can be anywhere on it, and the probability of landing exactly on an axis (where a coefficient is zero) is essentially nil. **Ridge shrinks everything and zeroes nothing.**

The L1 constraint region is a diamond — a cross-polytope — **with corners lying exactly on the axes**. Expanding elliptical contours are disproportionately likely to touch a corner first, because a corner protrudes.

And a corner is a sparse solution: at a corner of the L1 ball, all but one coordinate is exactly zero. **The sparsity comes from the shape of the constraint region**, and the two-dimensional picture of the circle and the diamond is worth drawing once by hand — after which lasso's behaviour is obvious rather than memorised.

Penalty	What it produces	When to use it
L2 (ridge)	Small, spread-out coefficients; nothing exactly zero.	Many weak, correlated features — the common case, and the sensible default. It shares weight among correlated features rather than choosing.
L1 (lasso)	Exact zeros , so the fit selects a subset of features.	When you want to know which features matter , or need a small model for deployment. With the caveat below.
Elastic net	Both penalties combined; selects correlated features as groups.	Correlated features where lasso's arbitrary choice is a problem — which is why it exists.
Early stopping	Implicit shrinkage toward the initialisation.	Any iterative model. Costs nothing and is frequently forgotten (Module 03 §3).
Dropout, noise, augmentation	Noise-based regularisation.	Neural networks — CSCE 636 Module 04.

The lasso caveat matters. Given several highly correlated features carrying the same information, **lasso selects one essentially arbitrarily and zeroes the rest** — and which one it picks can change with a small perturbation of the data. **So a lasso-selected feature set is not a stable finding about which variables matter**, and reporting it as one is a common and consequential error in applied work.

2 Cross-validation, done correctly

```
# WRONG -- and extremely common
X_scaled = StandardScaler().fit_transform(X)      # sees ALL data
cross_val_score(model, X_scaled, y, cv=5)
# the scaler learned the validation folds' mean and std.
# Optimistic, and silently so.

# ALSO WRONG, and much worse
X_sel = SelectKBest(k=20).fit_transform(X, y)      # sees ALL y
cross_val_score(model, X_sel, y, cv=5)
# selection used the HELD-OUT LABELS. On pure noise this can
# produce 80%+ "accuracy".

# RIGHT -- the pipeline IS the estimator, refit inside each fold
pipe = Pipeline([("scale", StandardScaler()),
                  ("select", SelectKBest(k=20)),
                  ("model", LogisticRegression())])
cross_val_score(pipe, X, y, cv=5)

# RULE: anything that LEARNS from data goes inside the pipeline.
```

The second case is worth running yourself on random noise. Generate a matrix of pure random features and random labels, select the twenty features most correlated with the labels across the whole dataset, then cross-validate. **You will get an accuracy far above chance on data containing no signal at all**, because the selection step found the features that happened to correlate with the labels *in the validation folds too*. It

is a two-minute demonstration and it is considerably more convincing than the warning.

THE THREE-WAY SPLIT, AND SPENDING THE TEST SET

Training set fits the parameters. Validation set chooses the hyperparameters. Test set estimates performance, once. The three roles are distinct and conflating any two of them produces an optimistic number.

Every time you look at the test set and then change something — anything — you have used it for selection. It has become a validation set, and its estimate of generalisation is now biased upward by an amount you cannot measure.

So the test set is spent once, at the very end, and the number is reported whatever it turns out to be. This requires discipline that notebooks actively undermine, which is why the project asks for the protocol to be committed before training.

With cross-validation the validation set is the folds themselves, and hyperparameter selection then requires *nested* cross-validation to produce an unbiased estimate — an outer loop for estimating performance and an inner loop for choosing hyperparameters within each outer training set. **This is rarely done because it is expensive, which means most reported cross-validation scores are mildly optimistic** — a small effect, usually, and worth knowing about when comparing against published numbers.

3 Hyperparameter search

Strategy	How it works	Character
Grid search	Every combination of a discrete grid over each hyperparameter.	Wasteful. If only two of five hyperparameters actually matter, a grid spends most of its budget varying the irrelevant ones while sampling the important ones coarsely.
Random search	Sample configurations at random from a distribution over each hyperparameter.	Beats grid search at the same budget — Bergstra and Bengio's result — because every trial samples a new value of <i>every</i> hyperparameter, so the important ones get fine coverage. Use this as the default.
Bayesian optimisation / TPE	Build a surrogate model of the response surface and sample where it is promising or uncertain.	Better per trial , with its own overhead. Worth it when a single training run is slow , which is the usual case in CSCE 636.
Successive halving / Hyperband	Train many configurations briefly, discard the worst, give the survivors more budget.	Very effective when training is expensive and early performance predicts final performance — which it usually does, but not always, and the exceptions are the configurations that start slowly and end well.
Hand tuning	A person looks at learning curves and changes one thing.	Underrated. A human reading a curve sees structure an automated search does not — 'this is diverging', 'this is underfitting' — and acts on the diagnosis of Module 03 rather than sampling blindly.

4 Choosing a model

MOST REPORTED DIFFERENCES ARE WITHIN THE NOISE

A validation score is an estimate, and it has a standard error. For accuracy p measured on m validation samples, the standard error is roughly $\sqrt{p(1-p)/m}$ — **on 1,000 samples at around 90% accuracy, that is about one percentage point.**

So 90.4% against 89.8% is not a result. It is the same quantity measured twice, and declaring a winner from it is reading noise. **A great deal of model selection, and a great deal of published comparison, consists of exactly this.**

Report the variability. The spread across cross-validation folds, or a bootstrap confidence interval, or a paired test across folds — anything that conveys how much the number would move if you had drawn a different sample.

And prefer the simpler model when they tie. It trains faster, is easier to explain, has fewer ways to break, and degrades more predictably under distribution shift (Module 13). **This is CSCE 735 Module 07's 'report the distribution rather than the mean' rule, arriving in a different subject** — and it is the same underlying point: **a single number conceals whether the difference you are acting on is real.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Hastie et al. — The Elements of Statistical Learning, chapters 3 and 7 (free PDF)	Ridge and lasso with the geometric argument of §1 , and the model selection theory of §2.
scikit-learn — cross-validation and model selection user guide (free)	The §2 pipeline discipline , with the leakage warning stated explicitly in the documentation — which is unusual and welcome.
Bergstra & Bengio — Random Search for Hyper-Parameter Optimization (free)	The §3 result . The figures showing why grid search wastes its budget are the whole argument.
Cawley & Talbot — On Over-fitting in Model Selection and Subsequent Selection Bias (free)	Why the inner-loop score is optimistic — the nested CV argument of §2, measured.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Fit ridge across a range of λ and plot every coefficient's path.
- 2 Do the same for lasso and **mark where each coefficient hits exactly zero**.
- 3 **Draw the L1 and L2 constraint regions in two dimensions** with loss contours, and mark the contact points.
- 4 **Demonstrate lasso's instability**: duplicate a feature with small noise, refit several times, and report which copy is selected.
- 5 **Run the noise demonstration**: random features, random labels, feature selection outside the fold. Report the accuracy you obtain.
- 6 Then put the selection inside a pipeline and report it again.
- 7 Implement nested cross-validation and compare the outer-loop estimate against the inner-loop one.
- 8 **Compare grid and random search at an equal trial budget** on a model with five hyperparameters of which two matter.
- 9 Implement successive halving and compare against random search at equal cost.
- 10 **Compute the standard error of your project's validation score** and decide whether your model differences are real.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the ridge and lasso objectives and say what each produces.
- 2 Explain geometrically why L1 gives sparsity.
- 3 Why is lasso's feature selection unstable, and what fixes it?
- 4 What must go inside the cross-validation fold, and why?
- 5 What happens if feature selection is done outside the fold?
- 6 State the three-way split and say what spending the test set means.
- 7 Why does random search beat grid search?
- 8 Why are most reported model differences not results?